

Migration Recommendations

Replacing an Orchestrator + Multi-Sub-Agent Pattern with Deep Agents + a Unified MCP Runtime

Prepared from the recommendations discussed in this chat

1. Executive Summary

The old codebase currently uses an orchestrator agent plus multiple persistent sub-agents. The recommended migration is not to recreate those sub-agents inside Deep Agents. Instead, the target architecture should use one Deep Agent, one unified MCP runtime, many MCP tools, and many task-specific skills.

- Keep the old application shell where useful: routes, request schemas, auth, config, lifecycle, and deployment structure.
- Replace the old orchestrator/sub-agent runtime with a single AgentEngine built around create_deep_agent(...).
- Move old sub-agent prompts into skills/<domain>/SKILL.md files.
- Move or wrap old sub-agent tools into MCP servers or MCP namespaces.
- Load all allowed tools through one unified MCPRuntime.
- Use one RequestObserver-style callback path to capture LLM calls, tool calls, token usage, and tool duration.

The high-level goal is:

```
one app runtime
one agent engine
one MCP runtime
many MCP tools
many skills
no persistent sub-agent graph
```

2. Current vs. Target Architecture

Old architecture

```
User request
-> API / app service
-> Orchestrator agent
    -> Sub-agent A
    -> Sub-agent B
    -> Sub-agent C
    -> each sub-agent owns its own prompt/tools/state
```

Target architecture

```
User request
-> API / app service
-> ChatSessionStore
-> AgentEngine
-> create_deep_agent(...)
-> Unified MCPRuntime.get_all_tools()
-> Core MCP gateway / BYOT gateway / mounted servers
-> Response + telemetry
```

The main architectural shift is moving from agent-level specialization to skill/tool-level specialization.

Old Pattern	New Pattern
Supervisor / orchestrator agent	A thin AgentEngine around one Deep Agent
Persistent worker/sub-agent	A skill folder plus MCP tools
Sub-agent prompt	skills/<domain>/SKILL.md
Sub-agent private tool list	MCP namespace or MCP server
Sub-agent router logic	Deep Agent chooses skills and MCP tools directly
Multiple agent states	One session state / one thread state

3. Recommended Migration Strategy

Do not perform a broad rewrite first. Migrate the runtime path in controlled layers.

1. Identify the old top-level agent entry point.
2. Bypass the old orchestrator/sub-agent router.
3. Add or port the unified MCPRuntime.
4. Load all MCP tools once at startup.
5. Create one Deep Agent with `create_deep_agent(...)`.
6. Point the old chat/API route to the new AgentEngine.
7. Convert old sub-agent prompts into Skills.
8. Convert old sub-agent tools into MCP servers or MCP namespaces.
9. Add callback-based observability.
10. Add AGENTS.md and memory only after the core runtime works.

4. File Categories That Will Likely Change

Because the old repo is not available yet, the filenames below are representative. The exact file names may differ, but these are the categories I would expect to touch.

Category	Likely Files	Expected Change
App startup / lifecycle	main.py, app.py, server.py, backend/main.py, api/main.py	Initialize Settings, MCPRuntime, one AgentEngine, session store, and observability. Remove startup wiring that builds several sub-agents.
Old orchestrator agent	agents/orchestrator.py, agents/supervisor.py, backend/orchestrator.py	Bypass or replace with a thin AgentEngine. Remove routing-to-sub-agent responsibilities.
Sub-agent files	agents/*_agent.py, subagents/*.py, workers/*.py	Do not convert these into Deep Agents subagents. Convert prompts to SKILL.md and tools to MCP.
Agent factory / registry	agents/factory.py, agents/registry.py, agent_builder.py	Replace multi-agent construction with one Deep Agent construction.
Tool registry / tool loader	tools/registry.py, tools/loader.py, backend/tools.py	Replace local per-agent tool lists with MCPRuntime.get_all_tools().
Prompt files	prompts/orchestrator.md, prompts/*_agent.md, prompt_templates.py	Move stable app-level guidance to system_prompt or AGENTS.md. Move sub-agent procedures to skills/<domain>/SKILL.md.
Config / settings	config.py, settings.py, backend/config.py, .env.example	Add MCP gateway URLs, skills dir, model settings, metrics path, and debug flags.
Session / memory	session.py, memory.py, conversation_store.py, state.py	Simplify to one session history per session_id. Avoid separate sub-agent histories.
Chat endpoint	routes/chat.py, api/chat.py, controllers/chat_controller.py	Replace orchestrator.run(...) with agent_engine.respond(...).
Observability	callbacks.py, tracing.py, metrics.py, telemetry.py	Move toward one RequestObserver that captures real LLM/tool events and tokens.
Dependencies	pyproject.toml, requirements.txt, uv.lock, Dockerfile	Add deepagents, FastMCP, MCP adapter packages, and observability dependencies as needed.
Tests	tests/test_orchestrator.py, tests/test_subagents.py, tests/test_agent_routing.py	Rewrite routing tests into Deep Agent, MCPRuntime, skills, and observability tests.

5. Minimal First-Pass File Set

If you want the smallest useful migration, I would focus on these first.

Existing files likely to update

```
backend/main.py
backend/config.py
backend/routes/chat.py or backend/main.py
backend/agents/orchestrator.py
backend/agents/factory.py
backend/tools/registry.py
pyproject.toml
.env.example
```

New files likely to add or port

```
backend/mcp_runtime.py
backend/agent.py
backend/session_store.py
backend/observability/request_observer.py
skills/
AGENTS.md
gateways/core_gateway.py
mcp_server/<domain>_server.py
```

6. How to Convert Old Sub-Agents

The old sub-agent files should not become Deep Agents subagents by default. They should usually become skills plus MCP tools.

Old Asset	New Asset
Old SQLAgent prompt	skills/sql/SKILL.md
Old SQLAgent tools	mcp_server/sql_server.py or sql namespace in the MCP gateway
Old ResearchAgent prompt	skills/research/SKILL.md
Old ResearchAgent tools	mcp_server/research_server.py or research namespace in the MCP gateway
Old ReportAgent prompt	skills/reporting/SKILL.md
Old ReportAgent tools	mcp_server/reporting_server.py or reporting namespace in the MCP gateway
Old MathAgent prompt	skills/math/SKILL.md
Old MathAgent tools	mcp_server/calculator_server.py or math namespace in the MCP gateway

Recommended skill directory layout:

```
skills/
  math/
    SKILL.md
  quote_count/
    SKILL.md
  sql/
    SKILL.md
  research/
    SKILL.md
  reporting/
    SKILL.md
```

7. Unified MCP Runtime Integration

The unified MCPRuntime should become the only tool-loading path. The Deep Agent should receive the final allowed tool list from MCPRuntime.

```
mcp_runtime = MCPRuntime(settings)
await mcp_runtime.start()

tools = await mcp_runtime.get_all_tools()

agent = create_deep_agent(
    model=model,
    tools=tools,
    system_prompt=system_prompt,
    skills=[skills_dir],
)
```

This replaces patterns like:

```
math_agent = create_agent(model=model, tools=math_tools, ...)
research_agent = create_agent(model=model, tools=research_tools, ...)
planner_agent = create_agent(model=model, tools=planner_tools, ...)
```

8. AgentEngine Target Shape

The new AgentEngine should be responsible only for model construction, tool loading, Deep Agent construction, invocation, and collecting observer metrics.

```
class AgentEngine:
    def __init__(self, settings: Settings, mcp_runtime: MCPRuntime) -> None:
        self.settings = settings
        self.mcp_runtime = mcp_runtime
        self.agent = None

    async def start(self) -> None:
        tools = await self.mcp_runtime.get_all_tools()

        self.agent = create_deep_agent(
            model=ChatOpenAI(
                model=self.settings.llm_model,
                base_url=self.settings.llm_base_url,
                api_key=self.settings.llm_api_key,
                temperature=self.settings.llm_temperature,
            ),
            tools=tools,
            system_prompt=self.settings.app_system_prompt,
            skills=[self.settings.skills_dir],
        )
```

9. Observability Recommendations

Your newer callback approach is still a good fit. Keep one observer per request and capture real events from the single Deep Agent runtime.

- Capture LLM call count from actual model callback events.
- Capture tool calls from actual tool callback events.
- Capture tool names, tool duration, success count, and error count.
- Capture token usage at the end of each LLM call when the provider returns usage metadata.
- Keep optional debug flags for printing full LLM context and token usage.

10. Config and Environment Settings

The migrated codebase will likely need settings like these:

```
LLM_MODEL=...
LLM_BASE_URL=...
LLM_API_KEY=...
LLM_TEMPERATURE=0.0

CORE_GATEWAY_URL=http://127.0.0.1:8090/mcp
BYOT_GATEWAY_URL=

SKILLS_DIR=backend/skills
METRICS_PATH=metrics/chat_metrics.jsonl

DEBUG_PRINT_LLM_CONTEXT=false
DEBUG_PRINT_LLM_TOKENS=false
```

If the old system has one model setting per sub-agent, remove those at first unless you explicitly need model specialization.

11. Target Project Structure

A cleaned-up target structure could look like this:

```
old_codebase/
  .env.example
  pyproject.toml
  AGENTS.md

backend/
  main.py
  config.py
  schemas.py
  session_store.py

agent/
  __init__.py
  engine.py

mcp/
  __init__.py
  runtime.py

observability/
  __init__.py
  request_observer.py
  metrics.py
  models.py
  otel.py

gateways/
  __init__.py
  core_gateway.py

mcp_server/
  __init__.py
  calculator_server.py
  quote_counter_server.py
  sql_server.py
  research_server.py
```

```
skills/  
  math/  
    SKILL.md  
  quote_count/  
    SKILL.md  
  sql/  
    SKILL.md  
  research/  
    SKILL.md
```

12. Tests to Rewrite or Add

Old Test Assumption	New Test Direction
Orchestrator routes math request to MathAgent	Deep Agent receives math prompt and calls math_add through MCPRuntime
Orchestrator routes research request to ResearchAgent	Deep Agent sees research skill and calls research MCP tools
Sub-agent has private tool list	MCPRuntime exposes allowed namespaced tools
Per-agent state is preserved	SessionStore preserves one session conversation history
Sub-agent telemetry exists	RequestObserver captures request-level LLM/tool/token metrics

13. Risks and Mitigations

Risk	Mitigation
Prompt/context size increases	Use token usage logging, trim history, and add summarization later.
Deep Agent calls extra tools	Add skill/system guidance: after a complete tool result, answer directly.
Skill discovery fails	Use startup debug prints for skills_dir and SKILL.md existence. Verify the prompt lists available skills.
Old sub-agent behavior is accidentally preserved	Do not convert old agents into Deep Agents subagents. Convert them into skills and MCP tools.
Tool names become confusing	Use MCP namespaces or clear prefixes: sql_run_query, research_search, math_add.
Observability becomes noisy	Tag built-in Deep Agents tools separately from business MCP tools.

14. Validation Checklist

- App starts and initializes one MCPRuntime.
- MCPRuntime loads all expected tools from the gateway.
- Only one main Deep Agent is created.
- No old orchestrator/sub-agent routing path is used for chat requests.
- Skills appear in the LLM context under Available Skills.
- A simple math request calls the MCP math tool once and returns the correct answer.
- A former sub-agent workflow is now handled by a skill plus MCP tools.
- RequestObserver records LLM calls, tool calls, token usage, and tools used.
- Same session_id preserves history; different session_id starts clean.
- Prompt tokens are monitored and session history is trimmed or summarized when needed.

15. Final Recommendation

Use the old codebase as the host application, but use your newer project as the runtime pattern. Remove the persistent multi-sub-agent layer. Use one Deep Agent with one unified MCP runtime. Convert old sub-agent prompts into skills and old sub-agent tools into MCP servers or namespaces. Keep one callback-based observability path so you can compare behavior, token usage, and tool usage cleanly.